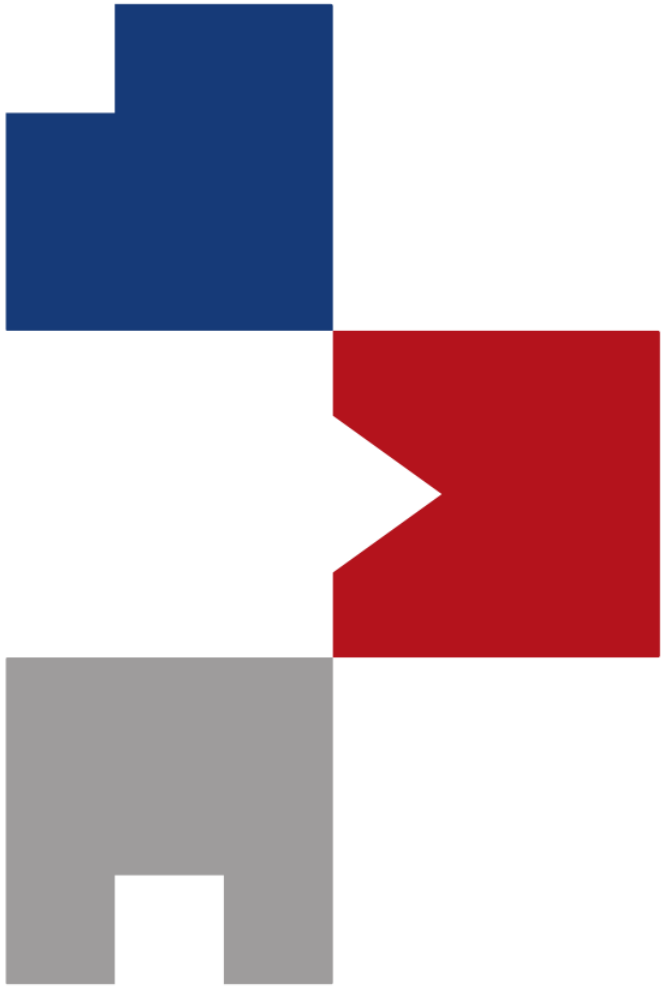
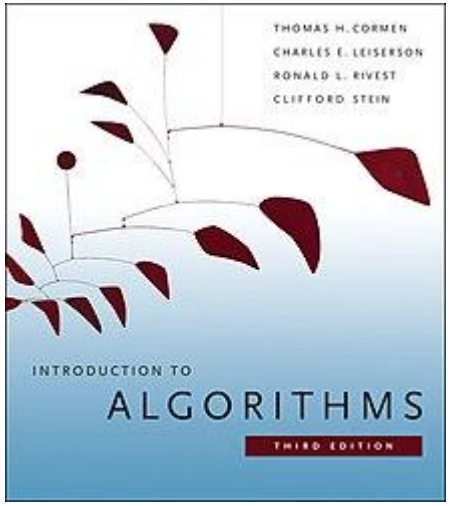
Note) Slides and example codes are available:
https://github.com/mint-lab/3dv_tutorial



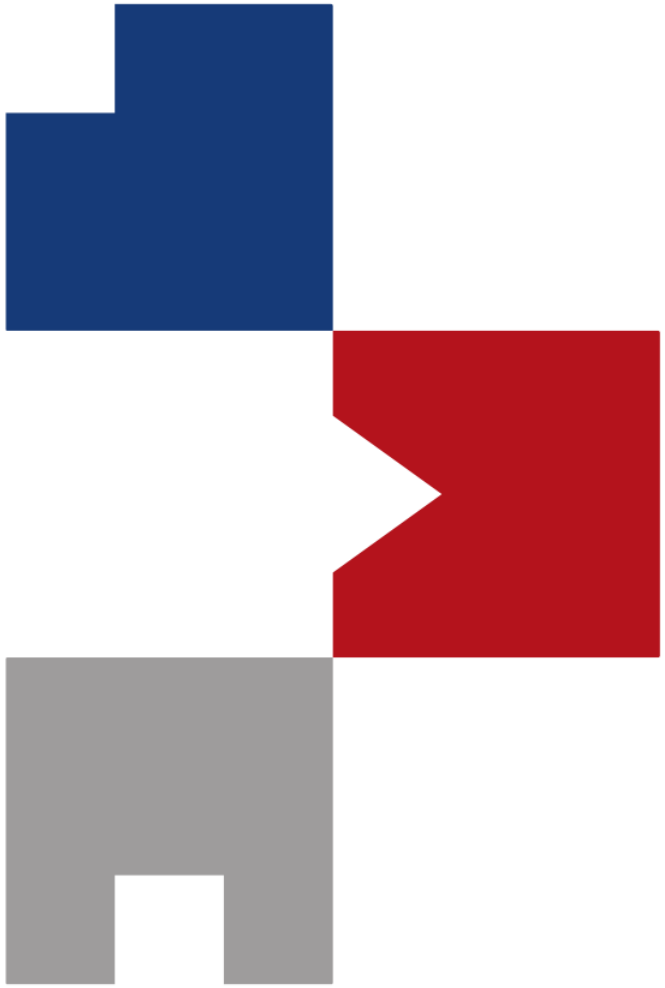
Introduction to 3D Vision



1312 pages

Sunglok Choi, Assistant Professor, Ph.D.
Dept. of Computer Science and Engineering, SEOULTECH
sunglok@seoultech.ac.kr | https://mint-lab.github.io/

Note) Slides and example codes are available:
https://github.com/mint-lab/3dv_tutorial

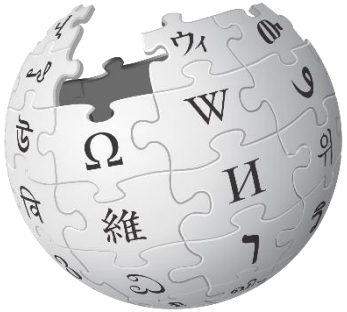


An Invitation ~~Introduction~~ to 3D Vision
: A Tutorial for Everyone

Sunglok Choi, Assistant Professor, Ph.D.
Computer Science and Engineering Department, SEOULTECH
sunglok@seoultech.ac.kr | https://mint-lab.github.io/

What is Computer Vision?

- **Computer vision is an interdisciplinary field that deals with how computers can be made to gain high-level understanding from digital images or videos.**
- From the perspective of engineering, it seeks to automate tasks that the human visual system can do.[1][2][3]
- "Computer vision is concerned with the automatic extraction, analysis and understanding of useful information from a single image or a sequence of images.
- It involves the development of a theoretical and algorithmic basis to achieve automatic visual understanding."[9]



Reference: Wikipedia

What is Computer Vision?

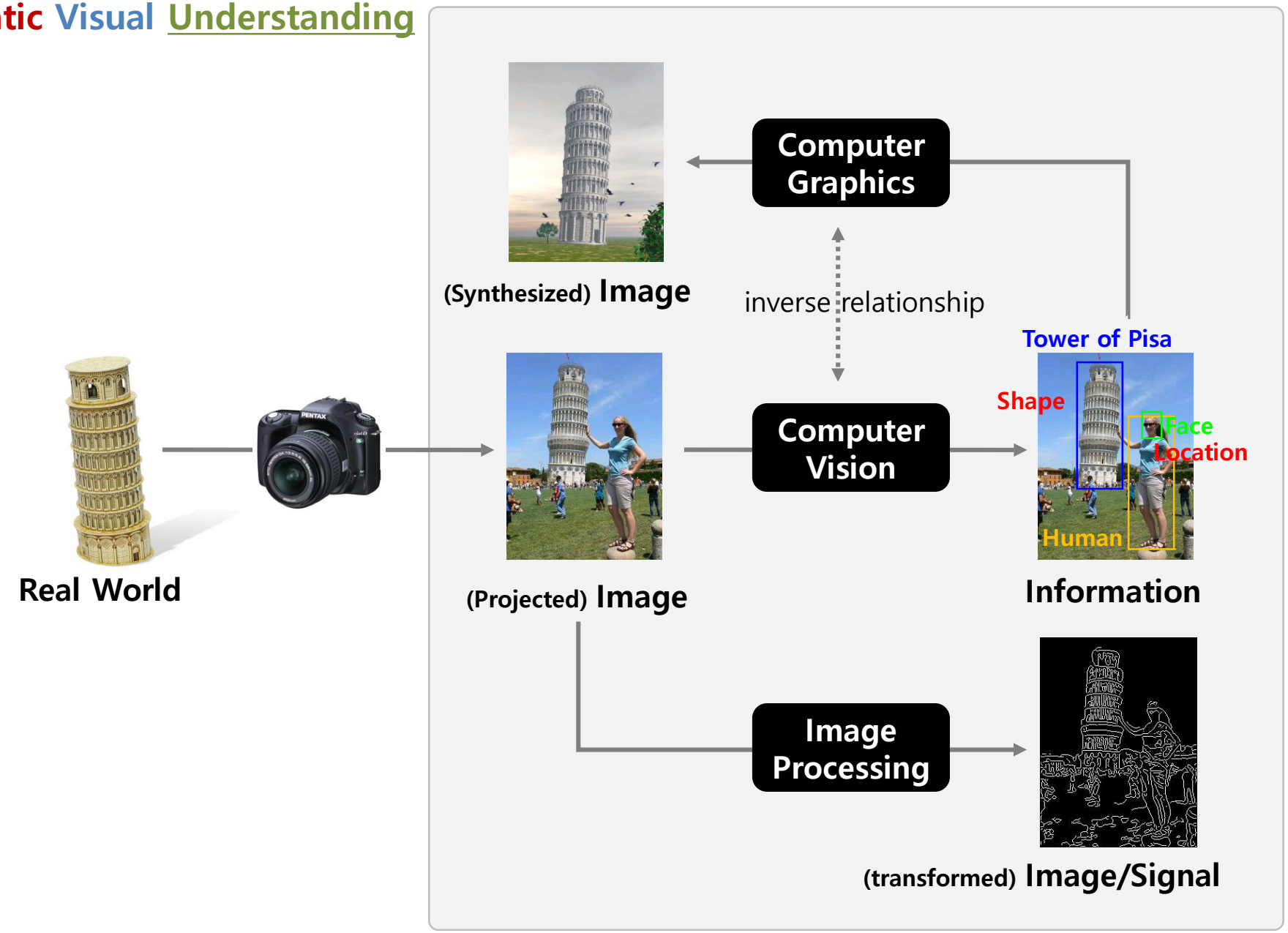
- **Computer vision is an interdisciplinary field that deals with how computers can be made to gain high-level understanding from digital images or videos.**
- From the perspective of engineering, it seeks to **automate** tasks that the human visual system can do.[1][2][3]
- "Computer vision is concerned with the **automatic** extraction, analysis and understanding of useful information from a single image or a sequence of images.
- It involves the development of a theoretical and algorithmic basis to achieve **automatic visual understanding**."[9]



Reference: Wikipedia

What is Computer Vision?

Automatic Visual Understanding



What is Computer Vision?

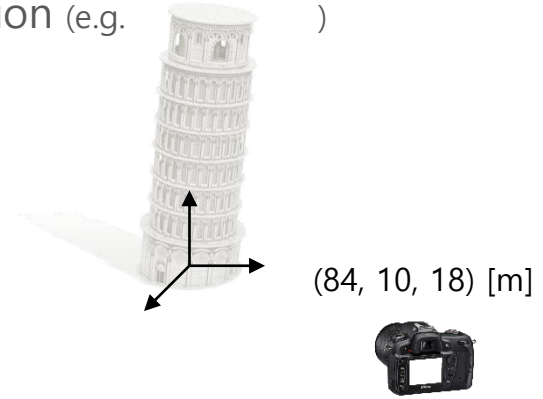
Computer Vision

What is it?

- Label (e.g. Tower of Pisa)
- Shape (e.g. )



Where am I?

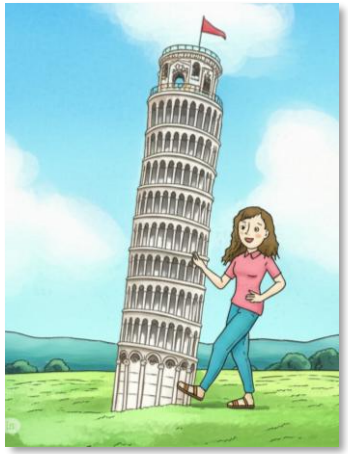
- Place (e.g. Piazza del Duomo, Pisa, Italy)
- Location (e.g. )

(84, 10, 18) [m]

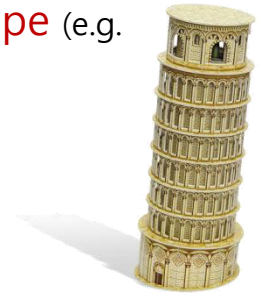
What is 3D Vision?

Automatic 3D Visual Perception

Computer Vision

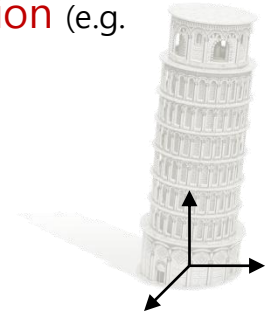


What is it?

- Label (e.g. Tower of Pisa)
- Shape (e.g. )



Where am I?

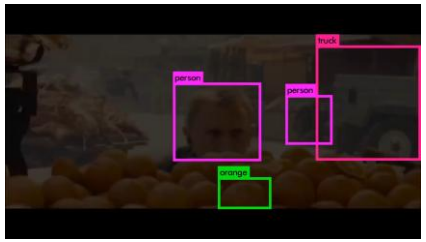
- Place (e.g. Piazza del Duomo, Pisa, Italy)
- Location (e.g. )

(84, 10, 18) [m]



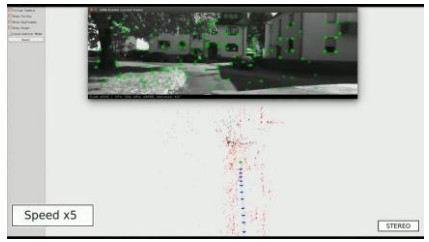
Generation Problems

Recognition Problems **vs.** **Reconstruction Problems**



YOLO v2 (2016)

Stanford CS231n:
[Deep Learning for Computer Vision]



ORB-SLAM2 (2016)

Stanford CS231A:
[Computer Vision,
From 3D Perception to 3D Reconstruction and Beyond]

What is 3D Vision?

Visual Geometry

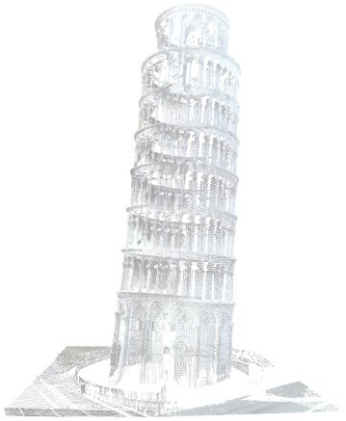
Multiple View Geometry

Geometric Vision

Photogrammetry



image



depth image, range data, point cloud, polygon mesh, ...



Perspective Camera

3D Vision vs. **3D Data Processing**



RGB-D Camera
(Stereo, Structured Light, ToF, Light Field)



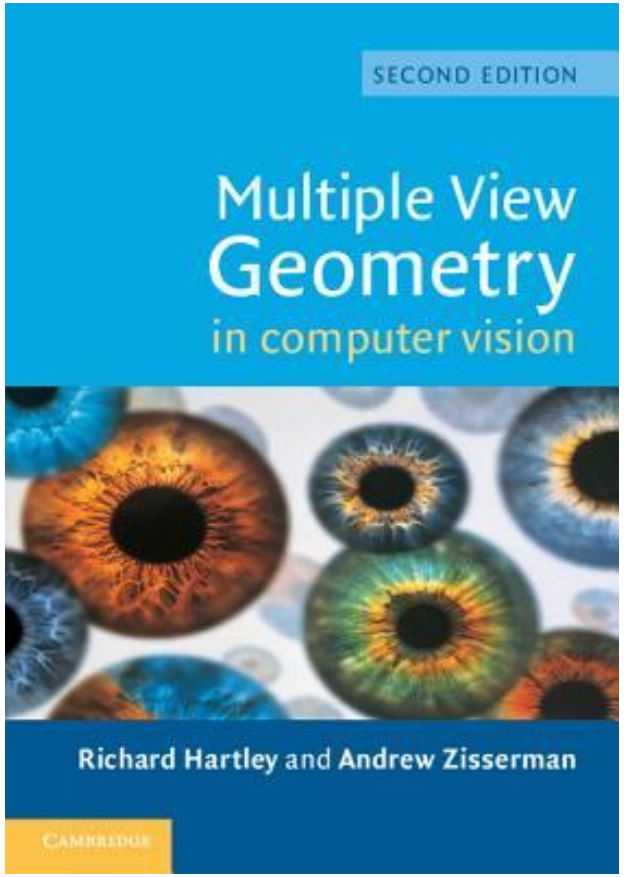
Omnidirectional Camera

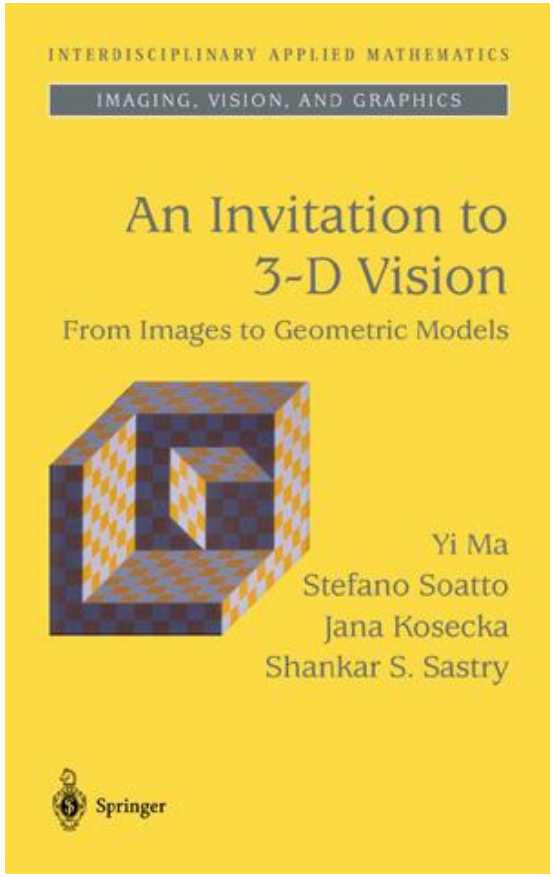


Range Sensor
(LiDAR, RADAR)

What is 3D Vision?

- Reference books





What is 3D Vision?

- All example codes are available at https://github.com/mint-lab/3dv_tutorial.
  - All example codes are mostly **less than 100 lines** and based on a **recent OpenCV version (4.0.0+)**.
  - Note) OpenCV (Open Source Computer Vision)

OpenCV v4.8.0 main modules:

- core. Core functionality
- imgproc. Image Processing
- imgcodecs. Image file reading and writing
- videoio. Video I/O
- highgui. High-level GUI
- video. Video Analysis
- calib3d. Camera Calibration and 3D Reconstruction
- features2d. 2D Features Framework
- objdetect. Object Detection
- dnn. Deep Neural Network module
- ml. Machine Learning
- flann. Clustering and Search in Multi-Dimensional Spaces
- photo. Computational Photography
- stitching. Images stitching
- gapi. Graph API

OpenCV v5.0.0-pre main modules:

- core. Core functionality
- imgproc. Image Processing
- imgcodecs. Image file reading and writing
- videoio. Video I/O
- highgui. High-level GUI
- video. Video Analysis
- 3d. 3d
- stereo. Stereo Correspondence
- features2d. 2D Features Framework
- calib. Camera Calibration
- objdetect. Object Detection
- dnn. Deep Neural Network module
- ml. Machine Learning
- flann. Clustering and Search in Multi-Dimensional Spaces
- photo. Computational Photography
- stitching. Images stitching
- gapi. Graph API

Motivation

- Building 3D vision background to understand papers

Preliminaries: Given an input video, we represent a scene as a collection of camera poses $\mathbf{T} \in \mathbb{SE}(3)^N$ and a set of square image patches $\mathbf{P}$ extracted from the video frames. Using $\mathbf{d}$ to represent inverse depth and $(\mathbf{x}, \mathbf{y})$ to represent pixel coordinates, we represent each patch as the $4 \times p^2$ homogeneous array

$$\mathbf{P}_k = \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{1} \\ \mathbf{d} \end{pmatrix} \qquad \mathbf{x}, \mathbf{y}, \mathbf{d} \in \mathbb{R}^{1 \times p^2} \tag{1}$$

where p is the width of the patch. We assume a constant depth for the full patch, meaning that it forms a fronto-parallel plane in the frame from which it was extracted. Letting i denote the index of the *source frame* of the patch, i.e. the frame from which which patch $\mathbf{P}_k$ was extracted, we can reproject the patch onto another frame j

$$\mathbf{P}'_{kj} \sim \mathbf{K}\mathbf{T}_j{\mathbf{T}_i}^{-1}\mathbf{K}^{-1}\mathbf{P}_k. \tag{2}$$

taking $\mathbf{K}$ to be the 4×4 calibration matrix

$$\mathbf{K} = \begin{pmatrix} f_x & 0 & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{3}$$

The pixel coordinates $\mathbf{x}' = (x', y')$ can be recovered by dividing by the third element. For the rest of the paper, we use the shorthand $\mathbf{P}'_{kj} = \omega_{ij}(\mathbf{T}, \mathbf{P}_k)$ to denote the reprojection of patch k onto frame j in terms of pixel coordinates.

Teed et al, "**D**eep **P**atch **V**isual **O**dometry", NeurIPS, 2023 Github

3. Method

Before delving into the details of our method, we introduce below the essential concept of pointmaps.

Pointmap. In the following, we denote a dense 2D field of 3D points as a *pointmap* $X \in \mathbb{R}^{W \times H \times 3}$. In association with its corresponding RGB image I of resolution $W \times H$, X forms a one-to-one mapping between image pixels and 3D scene points, *i.e.* $I_{i,j} \leftrightarrow X_{i,j}$, for all pixel coordinates $(i, j) \in \{1 \dots W\} \times \{1 \dots H\}$. We assume here that each camera ray hits a single 3D point, *i.e.* ignoring the case of translucent surfaces.

Cameras and scene. Given the camera intrinsics $K \in \mathbb{R}^{3\times3}$, the pointmap X of the observed scene can be straightforwardly obtained from the ground-truth depthmap $D \in \mathbb{R}^{W \times H}$ as $X_{i,j} = K^{-1}\left[iD_{i,j}, jD_{i,j}, D_{i,j}\right]^\top$. Here, X is expressed in the camera coordinate frame. In the following, we denote as $X^{n,m}$ the pointmap X^n from camera n expressed in camera m's coordinate frame:

$$X^{n,m} = P_m P_n^{-1} h\left(X^n\right) \tag{1}$$

with $P_m, P_n \in \mathbb{R}^{3\times4}$ the world-to-camera poses for images n and m, and $h : (x, y, z) \to (x, y, z, 1)$ the homogeneous mapping.

Wang et al, "**DUSt3R**: Geometric 3D Vision Made Easy", CVPR, 2024 Homepage

Motivation

- Building 3D vision background to understand papers

3.1 Formulation

We denote the input as $\mathcal{I} = \{\mathbf{I}_i\}_{i=1}^{N_v}$ with each $\mathbf{I}_i \in \mathbb{R}^{H\times W\times 3}$. For $N_v = 1$ this is a monocular image, and for $N_v > 1$ it represents a video or multi-view set. Each image has depth $\mathbf{D}_i \in \mathbb{R}^{H\times W}$, camera extrinsics $[\mathbf{R}_i \mid \mathbf{t}_i]$, and intrinsics $\mathbf{K}_i$. The camera can also be represented as $\mathbf{v}_i \in \mathbb{R}^9$ with translation $\mathbf{t}_i \in \mathbb{R}^3$, rotation quaternion $\mathbf{q}_i \in \mathbb{R}^4$, and FOV parameters $\mathbf{f}_i \in \mathbb{R}^2$. A pixel $\mathbf{p} = (u, v, 1)^\top$ projects to a 3D point $\mathbf{P} = (X, Y, Z, 1)^\top$ by

$$\mathbf{P} = \mathbf{R}_i\big(\mathbf{D}_i(u, v)\,\mathbf{K}_i^{-1}\mathbf{p}\big) + \mathbf{t}_i,$$

through which the underlying 3D visual space can be faithfully recovered.

Depth-ray representation. Predicting a valid rotation matrix $\mathbf{R}_i$ is challenging due to the orthogonality constraint. To avoid this, we represent camera pose implicitly with a per-pixel ray map, aligned with the input image and depth map. For each pixel $\mathbf{p}$, the camera ray $\mathbf{r} \in \mathbb{R}^6$ is defined by its origin $\mathbf{t} \in \mathbb{R}^3$ and direction $\mathbf{d} \in \mathbb{R}^3$: $\mathbf{r} = (\mathbf{t}, \mathbf{d})$. The direction is obtained by backprojecting $\mathbf{p}$ into the camera frame and rotating it to the world frame: $\mathbf{d} = \mathbf{R}\mathbf{K}^{-1}\mathbf{p}$. The dense ray map $\mathbf{M} \in \mathbb{R}^{H\times W\times 6}$ stores these parameters for all pixels. We do not normalize $\mathbf{d}$, so its magnitude preserves the projection scale. Thus, a 3D point in world coordinates is simply $\mathbf{P} = \mathbf{t} + \mathbf{D}(u, v) \cdot \mathbf{d}$. This formulation enables consistent point cloud generation by combining predicted depth and ray maps through element-wise operations.

Lin et al, "**Depth Anything 3**: Recovering the Visual Space from Any Views", arXiv, 2025 Homepage

C NDC ray space derivation

We reconstruct real scenes with "forward facing" captures in the normalized device coordinate (NDC) space that is commonly used as part of the triangle rasterization pipeline. This space is convenient because it preserves parallel lines while converting the z axis (camera axis) to be linear in disparity.

Here we derive the transformation which is applied to rays to map them from camera space to NDC space. The standard 3D perspective projection matrix for homogeneous coordinates is:

$$M = \begin{pmatrix} \frac{n}{r} & 0 & 0 & 0 \\ 0 & \frac{n}{t} & 0 & 0 \\ 0 & 0 & \frac{-(f+n)}{f-n} & \frac{-2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix} \tag{7}$$

where n, f are the near and far clipping planes and r and t are the right and top bounds of the scene at the near clipping plane. (Note that this is in the convention where the camera is looking in the $-z$ direction.) To project a homogeneous point $(x, y, z, 1)^\top$, we left-multiply by M and then divide by the fourth coordinate:

$$\begin{pmatrix} \frac{n}{r} & 0 & 0 & 0 \\ 0 & \frac{n}{t} & 0 & 0 \\ 0 & 0 & \frac{-(f+n)}{f-n} & \frac{-2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{n}{r}x \\ \frac{n}{t}y \\ \frac{-(f+n)}{f-n}z - \frac{-2fn}{f-n} \\ -z \end{pmatrix} \tag{8}$$

$$\text{project} \rightarrow \begin{pmatrix} \frac{n}{r}\frac{x}{-z} \\ \frac{n}{t}\frac{y}{-z} \\ \frac{(f+n)}{f-n} - \frac{2fn}{f-n}\frac{1}{-z} \end{pmatrix} \tag{9}$$

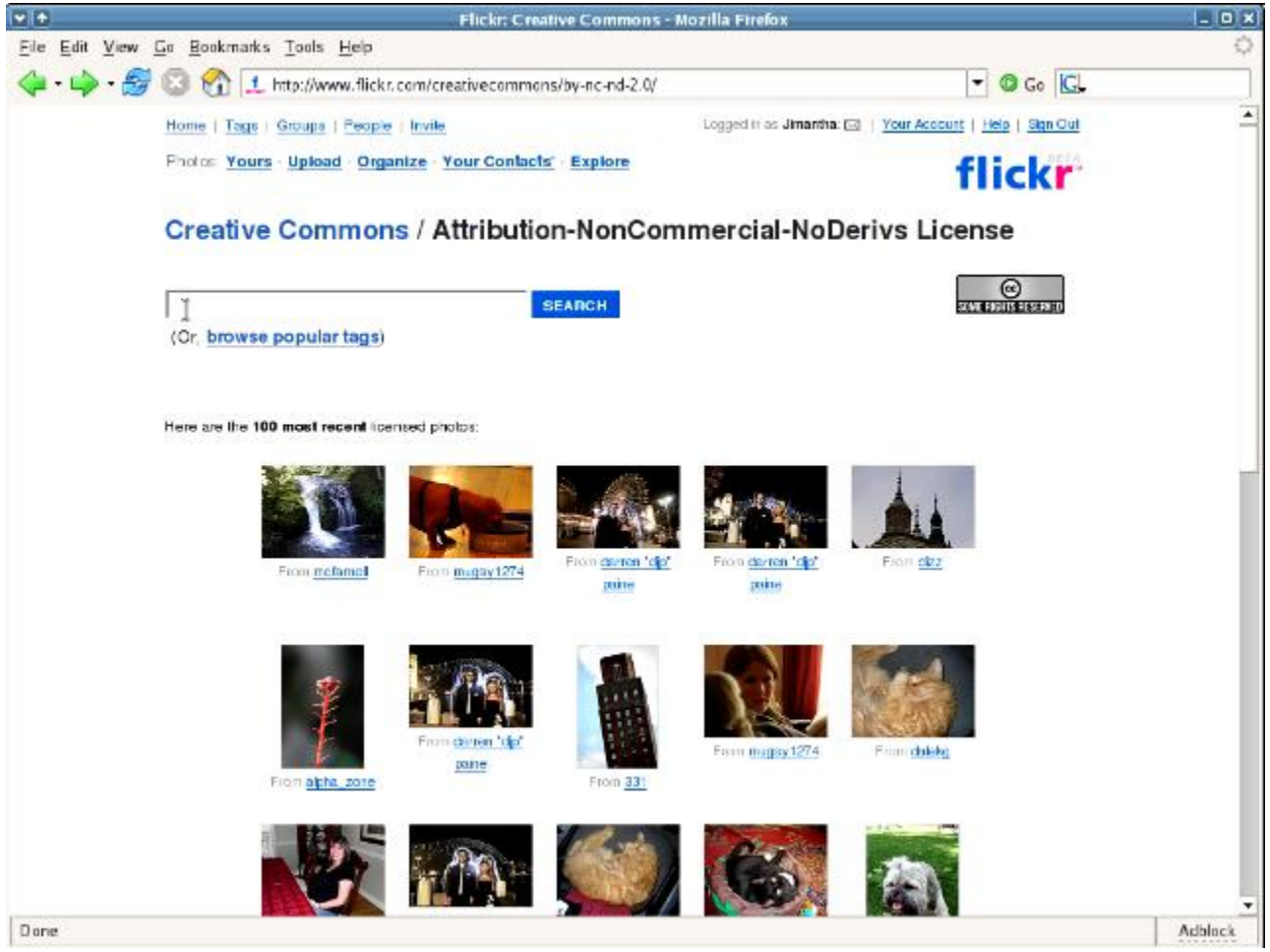
The projected point is now in normalized device coordinate (NDC) space, where the original viewing frustum has been mapped to the cube $[-1, 1]^3$.

Our goal is to take a ray $\mathbf{o} + t\mathbf{d}$ and calculate a ray origin $\mathbf{o}'$ and direction $\mathbf{d}'$ in NDC space such that for every t, there exists a new t' for which $\pi(\mathbf{o} + t\mathbf{d}) = \mathbf{o}' + t'\mathbf{d}'$ (where π is projection using the above matrix). In other words, the projection of the original ray and the NDC space ray trace out the same points (but not necessarily at the same rate).

Midenhall et al, "**NeRF**: Representing Scenes as Neural Radiance Fields for View Synthesis", ECCV, 2020 Homepage

Applications) Photo Browsing

- Photo Tourism (2006)

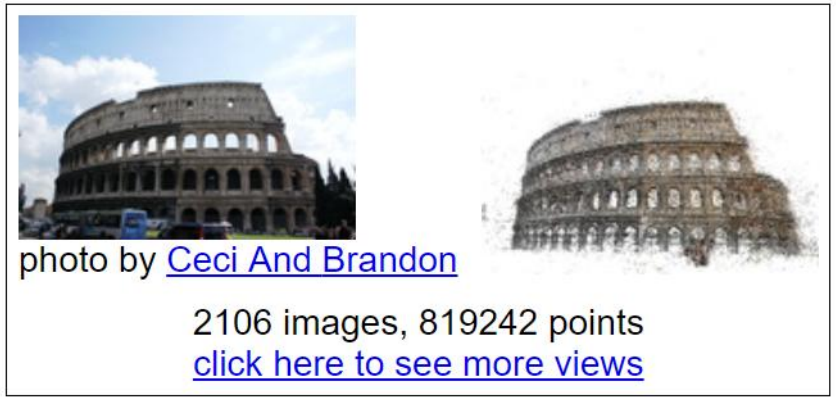




Reference: Snavely et al., "Photo Tourism: Exploring Photo Collections in 3D", SIGGRAPH, 2006 Homepage

Applications) 3D Reconstruction

- Building Rome in a Day (2009)









Reference: Agarwal et al., "Building Rome in a Day", ICCV, 2009 Homepage

Applications) 3D Reconstruction

- Building Rome in a Day (2009)



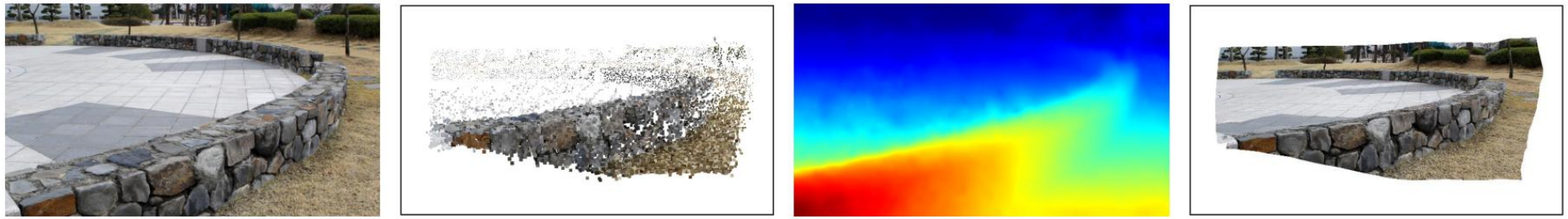
The Old City of Dubrovnik

YouTube

Reference: Agarwal et al., "Building Rome in a Day", ICCV, 2009 Homepage

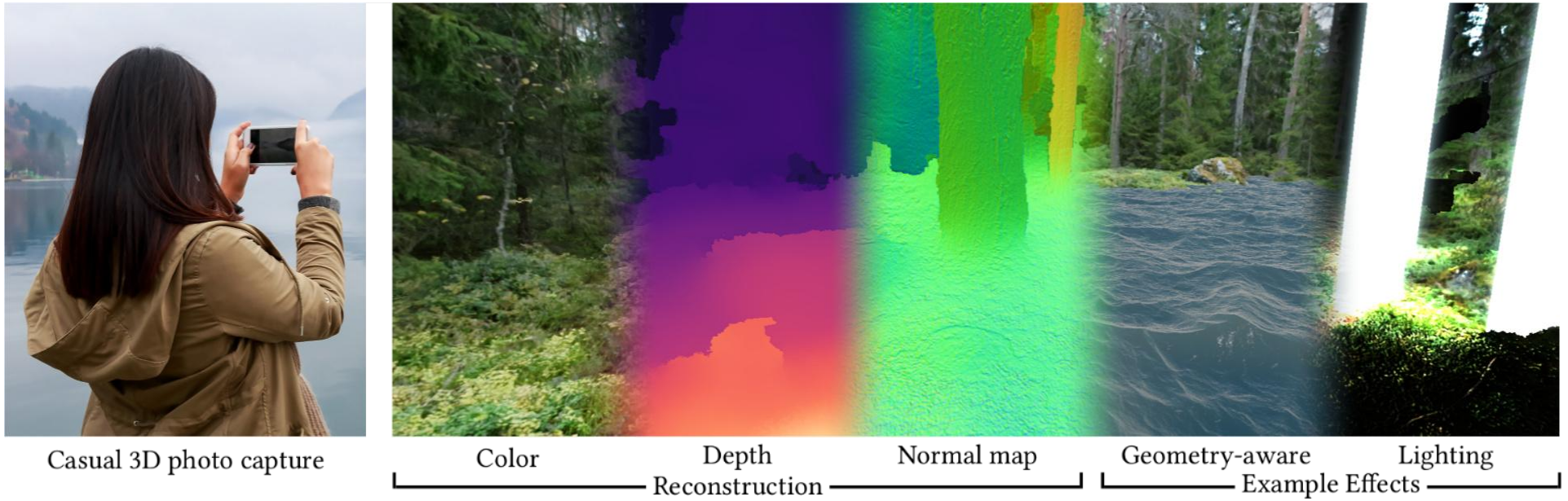
Applications) Depth Estimation from Cellular Phones

- Structure from Small Motion (SfSM; 2015)



(a) Reference images (b) SfSM results (c) Depth maps (d) Our 3D meshes

- Casual 3D Photography (2017)

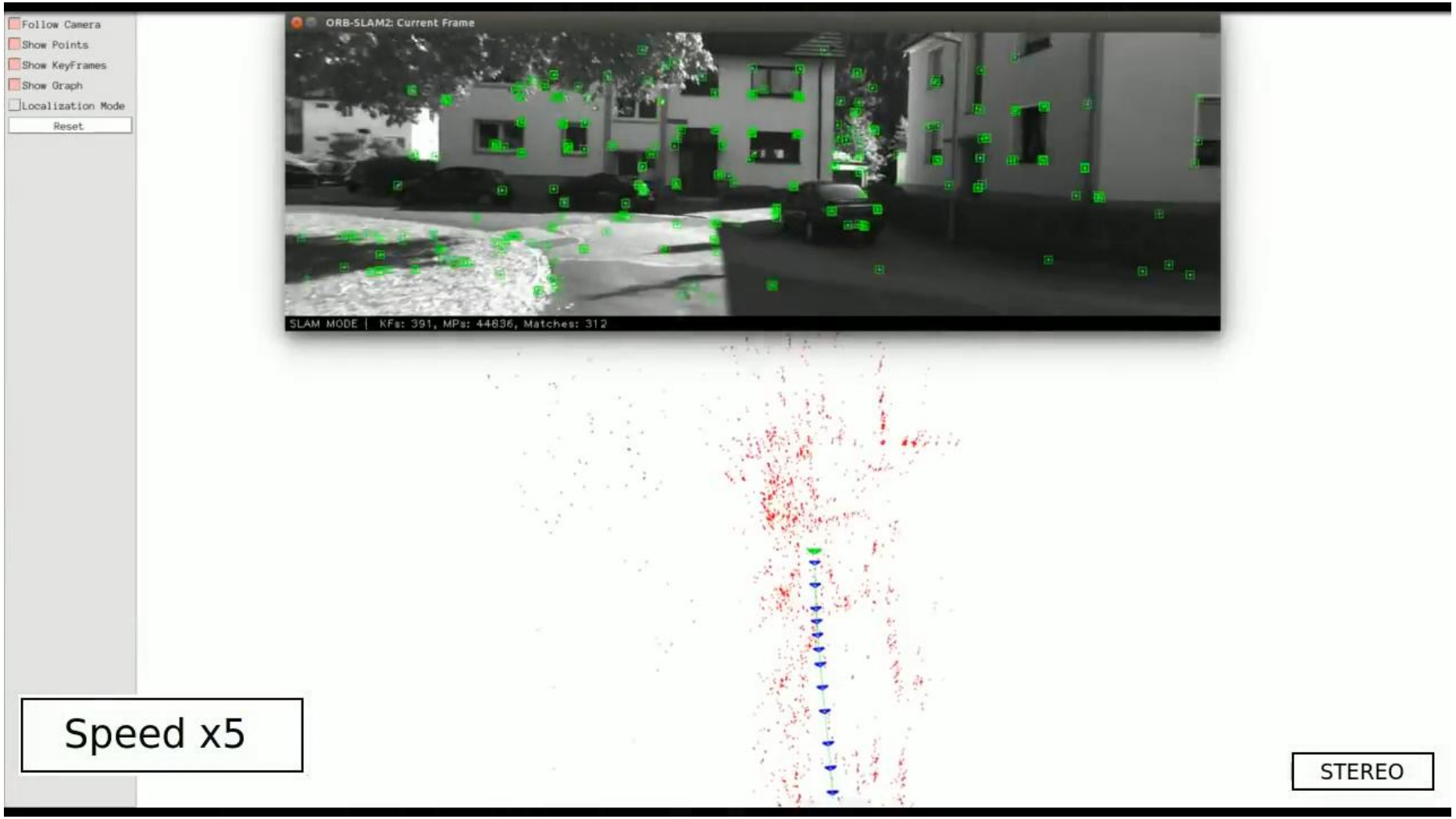


Casual 3D photo capture

Reference: Im et al., "High Quality Structure from Small Motion for Rolling Shutter Cameras", ICCV, 2015
Reference: Hedman et al., "Casual 3D Photography", SIGGRAPH Asia, 2017

Applications) Real-time Visual SLAM

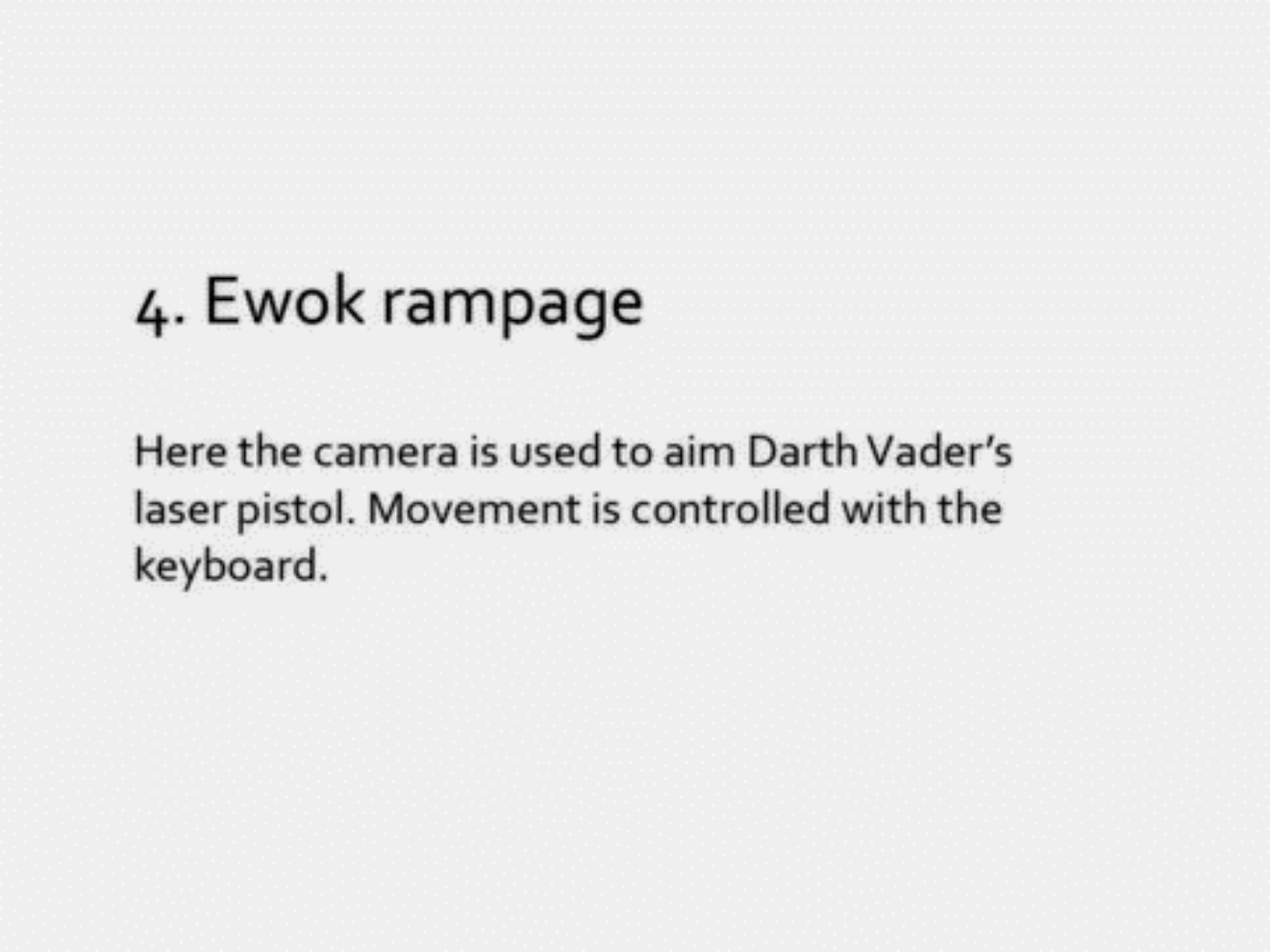
- ORB-SLAM (2014)



Reference: Mur-Artal et al., "ORB-SLAM: A Versatile and Accurate Monocular SLAM System", T-RO, 2015 Github

Applications) Augmented Reality

- PTAM: Parallel Tracking and Mapping (2007)



Reference: Klein and Murray, "Parallel Tracking and Mapping for Small AR Workspaces", ISMAR, 2007

Applications) Virtual Reality

- Oculus Quest (2019)



Image: TechSpot

Applications) Mixed Reality

- Microsoft HoloLens 2 (2019)
  - Head tracking: 4 x visible light cameras

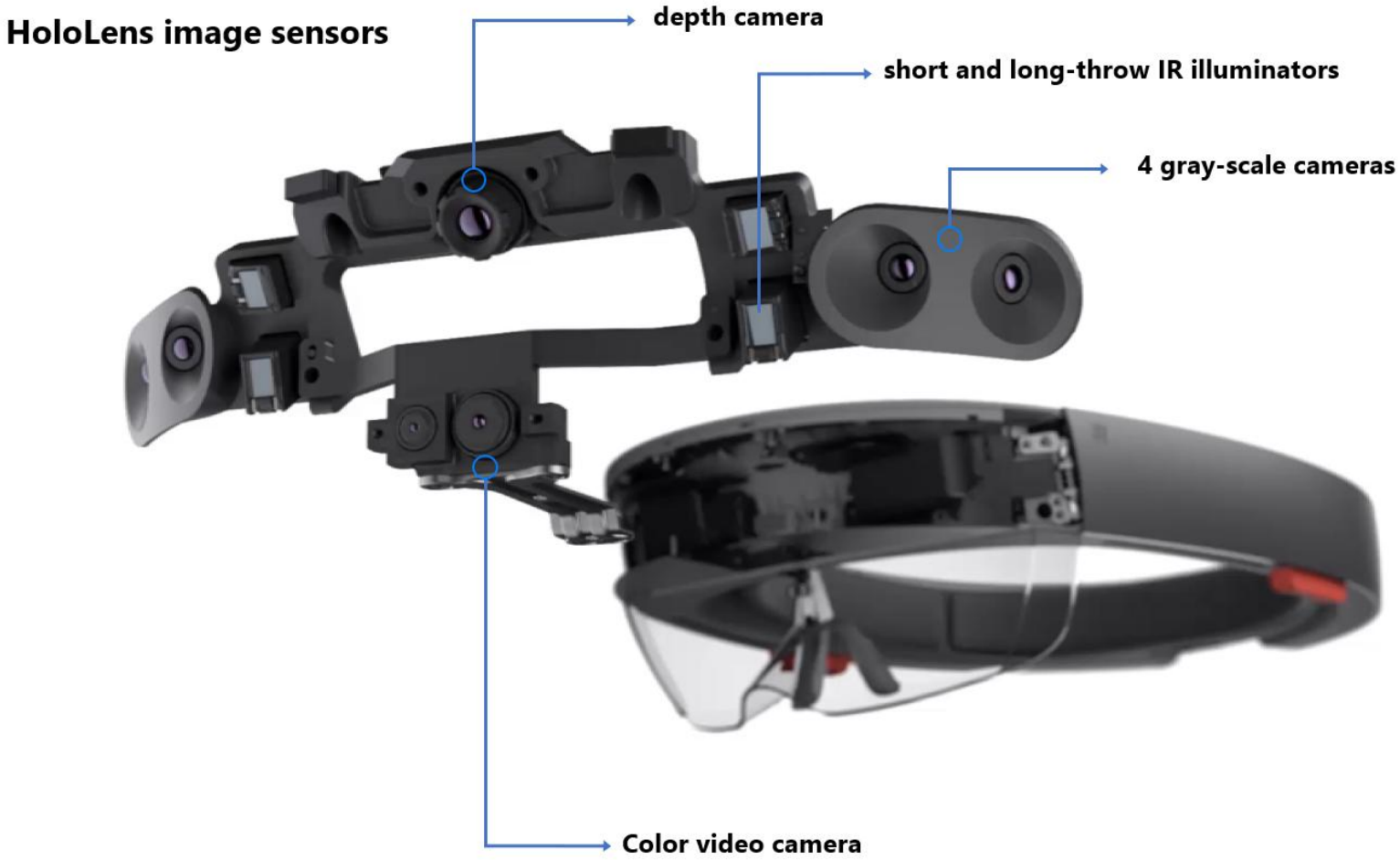


Image: SlashGear

Note) Slides and example codes are available:
https://github.com/mint-lab/3dv_tutorial

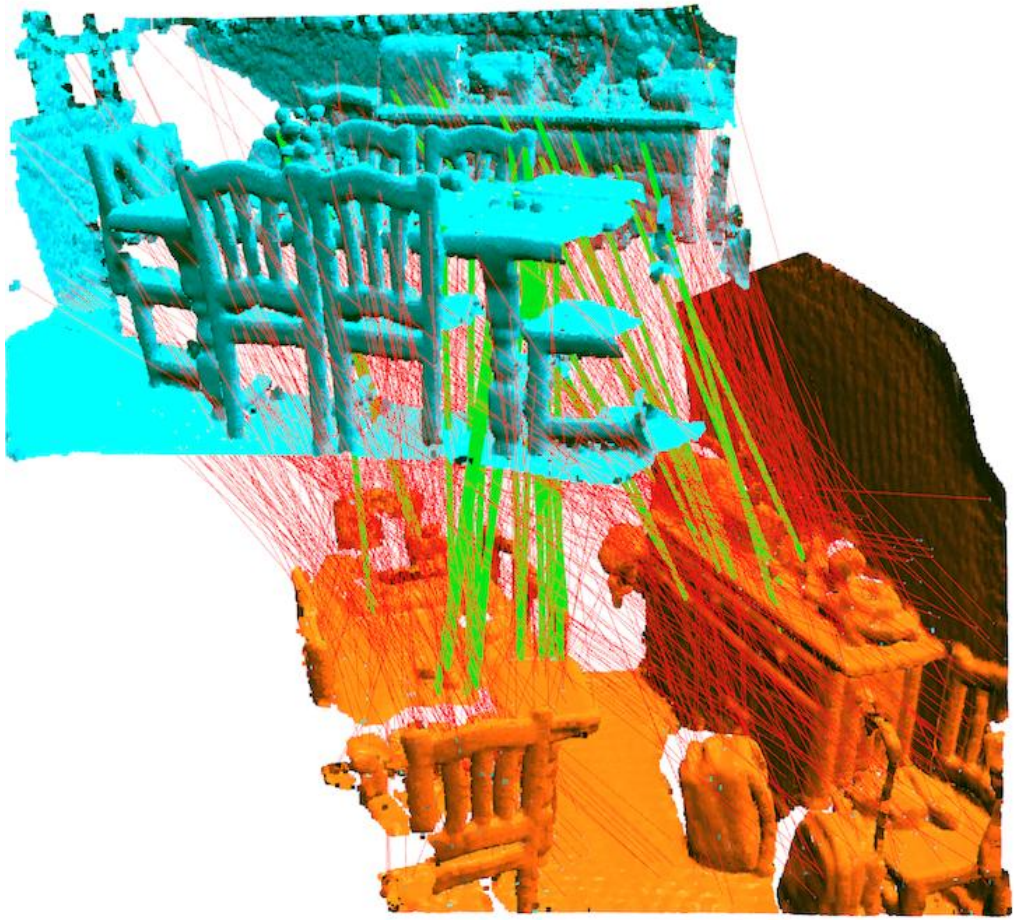
Summary) Introduction

- **What is Computer Vision?**
- **What is 3D Vision?**
  - What? Recognition problems vs. Reconstruction problems
    - Note) Generation problems vs. Reconstruction problems
  - Why? Applications

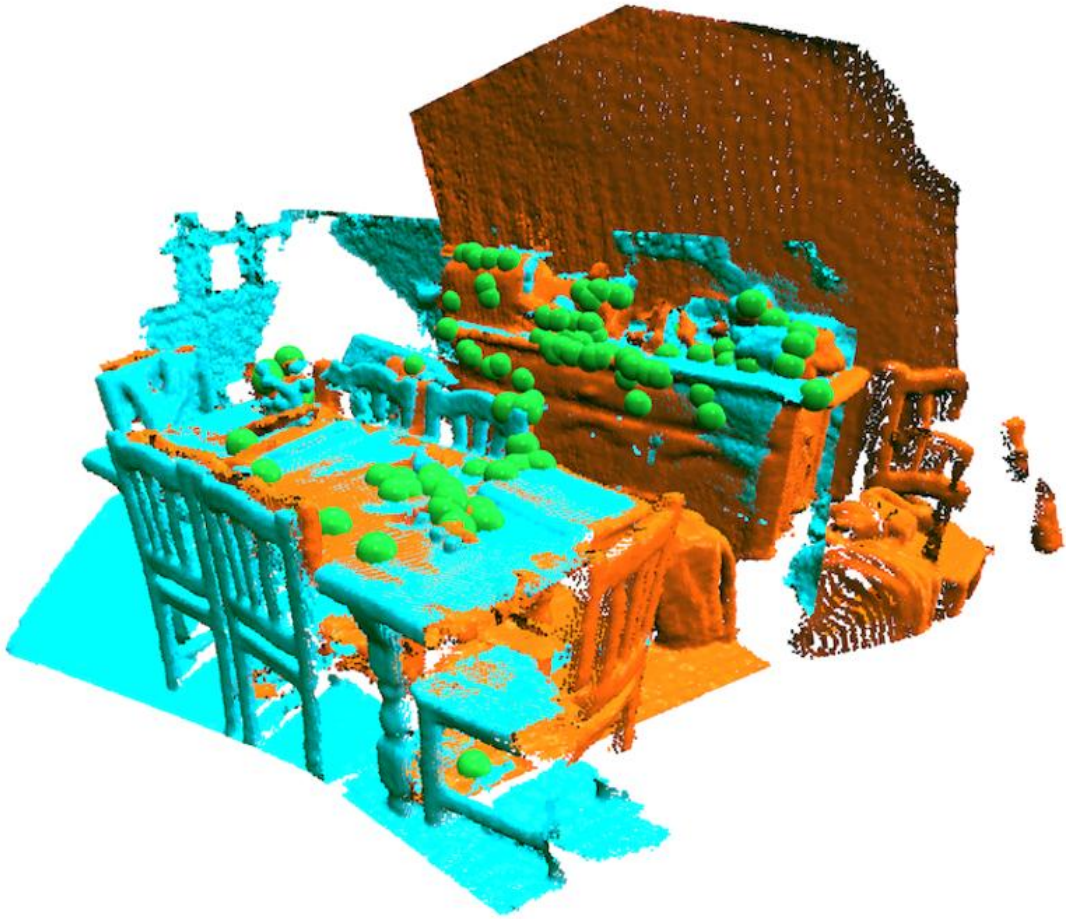
Next Topics

- **Single-view Geometry**
- **Two-view Geometry**
- **Solving Equations**
- **Finding Correspondences**
- **Multi-view Geometry**
- **Special Topic) Bayesian Filtering**
- **Visual SLAM and Odometry**
- **3D Representations**

3D Data Processing: 3D Registration



Correspondences



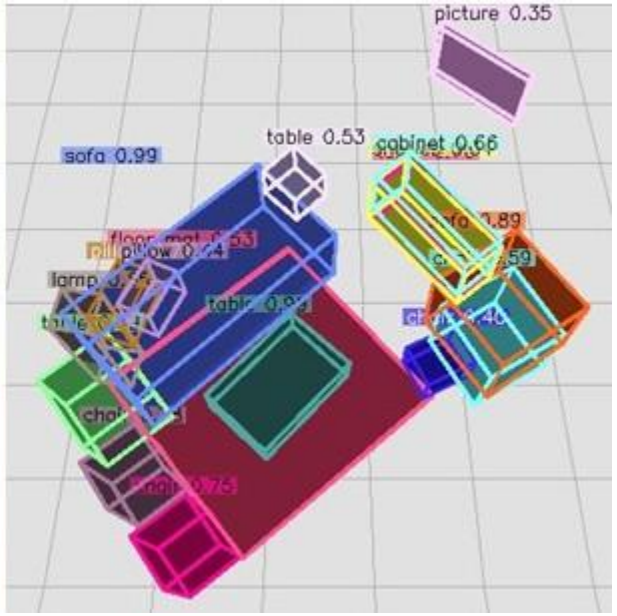
TEASER++ Alignment

Image: Yang et al., "TEASER: Fast and Certifiable Point Cloud Registration", T-RO, Vol. 37, No. 2, 2021 Github DOI

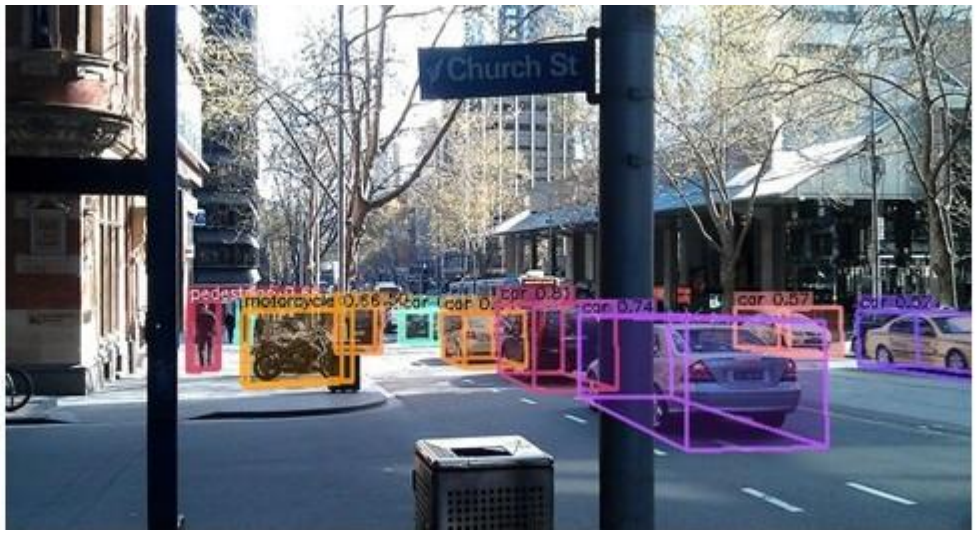
3D Data Processing: 3D Object Detection











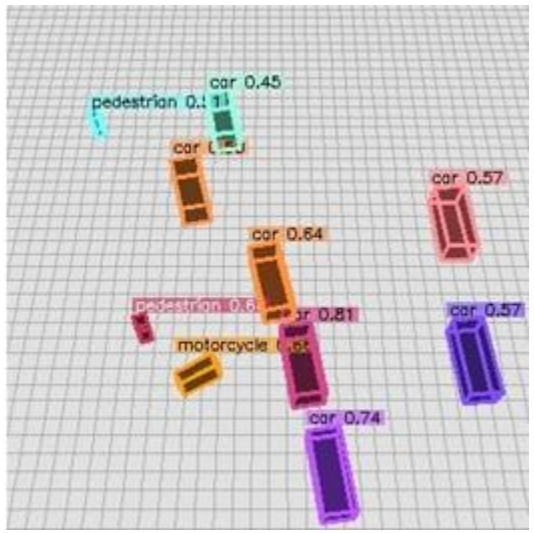


Image: Brazil et al., "Omni3D: A Large Benchmark and Model for 3D Object Detection in the Wild", CVPR, 2023 Homepage

3D Data Processing: 3D Semantic Segmentation

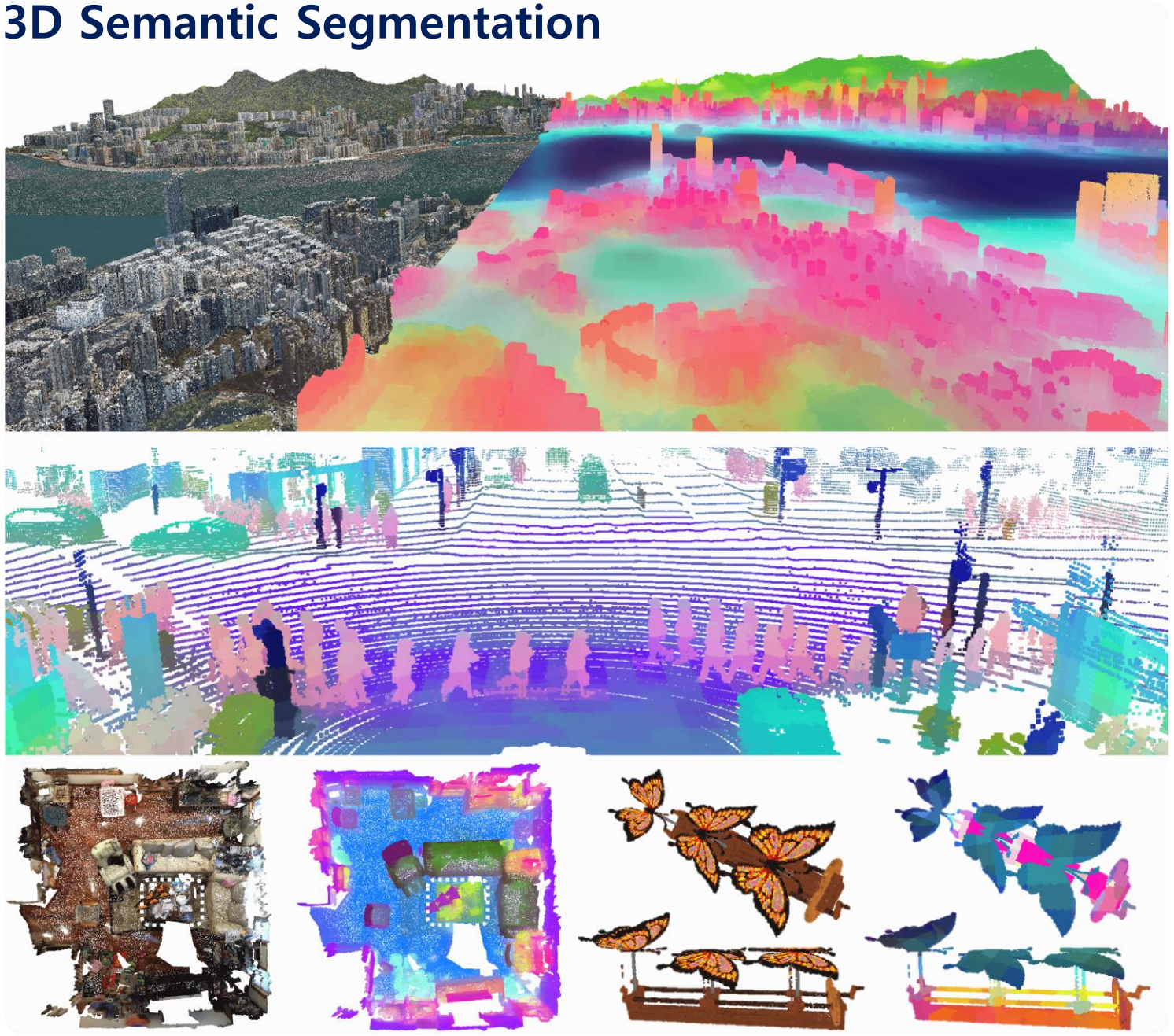


Image: Zhang et al., "Utonia: Toward One Encoder for All Point Clouds", arXiv, 2026 Homepage